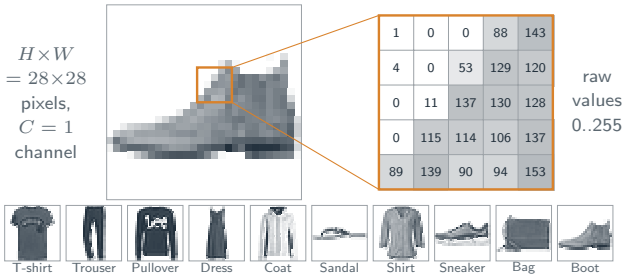
Convolutional Neural Networks from the Ground Up

Kernels, feature maps, pooling, backpropagation and training, with runnable code

---

A 20-slide tutorial • companion code: `conv_numpy.py`, `cnn.py`, `train.py`, `experiments.py`, `walkthrough.py`

Images are tensors, and why an MLP is the wrong tool for them



- **Image** = tensor of shape (C, H, W); colour has $C = 3$ (R, G, B). A batch is (N, C, H, W).
- **Scaling**: divide by 255, then standardise with the training-split mean and std: $x' = (x/255 - 0.286)/0.354$. The patch centre 137 becomes $+0.71$, black -0.81.
- **Labels**: class index $0..9$ in the order above. Data: Fashion-MNIST [13], 10,000 train / 2,000 validation / 10,000 test images (drawn dark-on-white).

A dense layer gives every unit its own weight per pixel. Two problems:

- **Too many weights**: image area times layer width.

first layer	$28 \times 28 \times 1$	$224 \times 224 \times 3$
inputs	784	150,528
dense, 1,000 units	785,000	150,529,000
conv, 64 filters 3×3	640	1,792

- **No notion of space.** After flattening, pixel 0 and pixel 1 are as unrelated as pixel 0 and pixel 783. A boot moved two pixels sideways is a new pattern to be learned again.

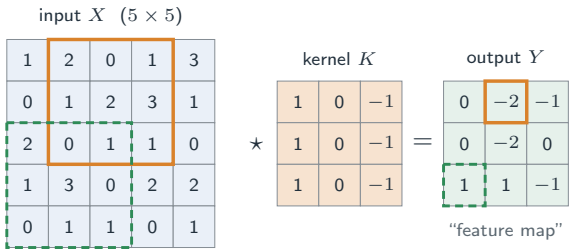
Measured: scramble the pixels

Apply one fixed random permutation to the 784 pixel positions of every image; retrain (seed 0).

	original	scrambled
MLP $784 \to 128 \to 64 \to 10$	84.8%	84.9%
CNN (slide 11)	88.0%	83.3%

The MLP does not care: it never used the layout. The CNN loses 4.7 points: its advantage *is* the assumption that neighbours belong together.

The convolution: slide a small kernel over the image



Orange window, $Y_{0,1}$: multiply the kernel entry by entry with the patch $\left[\begin{smallmatrix} 2 & 0 & 1 \\ 1 & 2 & 3 \\ 0 & 1 & 1 \end{smallmatrix}\right]$ and add up:

$$Y_{0,1} = (2+1+0)\cdot 1 + (0+2+1)\cdot 0 + (1+3+1)\cdot(-1) = 3 - 5 = \textbf{-2}$$

Green window, $Y_{2,0}$: $(2+1+0) - (1+0+1) = 3 - 2 = \textbf{1}$.

This kernel computes "left column minus right column": it responds where brightness changes from left to right. Nine multiplications per output value, the same nine weights everywhere.

Definition (one channel, no padding, stride 1)

$$Y_{i,j} = b + \sum_{u=0}^{k-1}\sum_{v=0}^{k-1} K_{u,v}\, X_{i+u,\, j+v}$$

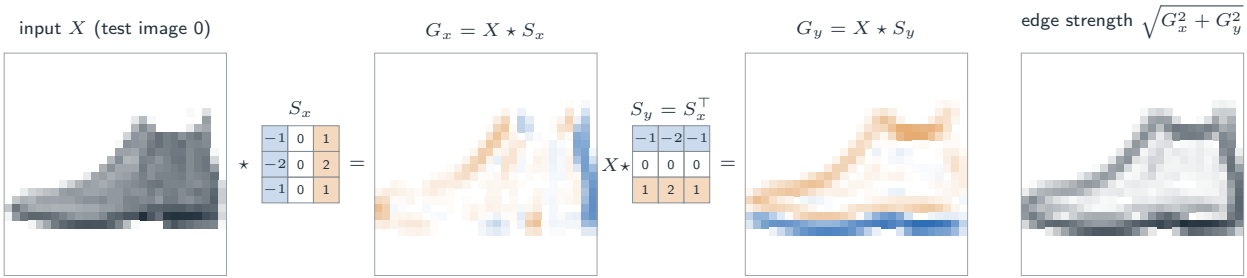
k: kernel size; b: one bias per kernel. Output size $n - k + 1$: here $5 - 3 + 1 = 3$.

- The **kernel** (or *filter*) is a small array of weights. It is the thing that is learned.
- Each output value is a dot product: it is large where the patch looks like the kernel.
- An output unit sees only a $k \times k$ patch: **local connectivity**. The same kernel is used at every position: **weight sharing** (slide 8).

A note on the name

Mathematicians call this **cross-correlation**, written $X \star K$. True convolution $X * K$ flips the kernel first, $K_{u,v} \to K_{k-1-u,\, k-1-v}$. Deep-learning libraries do not flip. Since K is learned, the difference is only a relabelling of the weights [14].

Kernels are feature detectors: Sobel edge filters on a real image



Computed by `conv2d` in `conv_numpy.py`, padding 1 (experiment B). Grey: darker = larger. Orange = positive, blue = negative response; full scale ± 3.94 for pixels in $[0, 1]$.

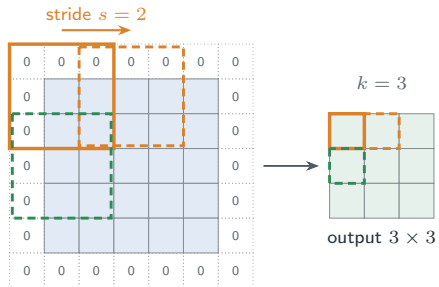
- S_x compares right with left: it fires on **vertical** edges (back of the shaft, toe). S_y compares below with above: **horizontal** edges (sole, top line).
- The sign gives the direction: dark→bright is positive.

By hand: a step edge

Rows $0\ 0\ 1\ 1\ 1$ (three identical rows): $X \star S_x = (\mathbf{4}, \mathbf{4}, 0)$: large at the step, zero on the flat part. $X \star S_y = (0, 0, 0)$.

- Each kernel sums to 0, so a constant region gives exactly 0: 63% of the last panel is below 5% of its maximum.
- These kernels were designed by hand in the 1960s [17]. A CNN **learns** its kernels; slide 18 shows ours.

Padding and stride: controlling the output size



$n = 5$, padding $p = 1$: 7×7 after padding

Output size along one axis

$$n_{\text{out}} = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1$$

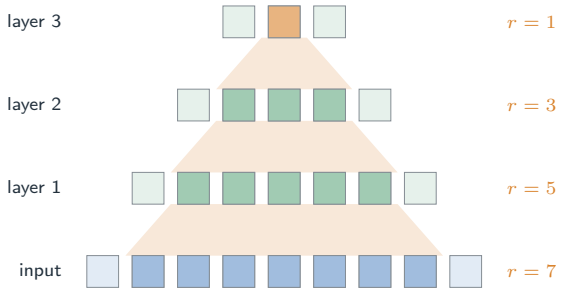
n input size, k kernel size, p padding on each side, s stride. Drawing: $\lfloor (5 + 2 - 3)/2 \rfloor + 1 = \mathbf{3}$.

- **Padding** p: surround the image with p rings of zeros. Without it every layer shrinks the map by $k - 1$ and border pixels are used less often than central ones. "Same" padding $p = (k - 1)/2$ keeps the size (odd k, $s = 1$).
- **Stride** s: move the window s pixels at a time. The output is about s times smaller per axis, so s^2 times cheaper. A strided convolution is a learned way of downsampling.
- The floor: last rows and columns are ignored when the window does not fit.

n	k	p	s	$\lfloor (n + 2p - k)/s \rfloor + 1$	where
5	3	0	1	$\lfloor 2 \rfloor + 1 = 3$	slide 3
5	3	1	1	$\lfloor 4 \rfloor + 1 = 5$	"same"
28	5	2	1	$\lfloor 27 \rfloor + 1 = 28$	our conv1
14	5	0	1	$\lfloor 9 \rfloor + 1 = 10$	our conv2
28	2	0	2	$\lfloor 13 \rfloor + 1 = 14$	our pool1
8	3	0	2	$\lfloor 2.5 \rfloor + 1 = 3$	last column unused
224	7	3	2	$\lfloor 111.5 \rfloor + 1 = 112$	ResNet stem [6]

The same formula covers pooling (slide 9). More cases (dilation, transposed convolution): [11].

Receptive field: how much of the image one unit can see



a 1-D cut through three stacked convolutions, $k = 3$, stride 1

The **receptive field** of a unit is the set of input pixels that can influence it. One 3×3 layer sees 3 pixels across; each further layer adds $k - 1 = 2$.

Recurrence (per axis), from the input upward

$$r_\ell = r_{\ell-1} + (k_\ell - 1)\, j_{\ell-1}, \qquad j_\ell = j_{\ell-1}\, s_\ell, \qquad r_0 = j_0 = 1$$

j ("jump") is the distance in input pixels between neighbouring units of a layer. Stride and pooling multiply j, so every later layer grows r faster.

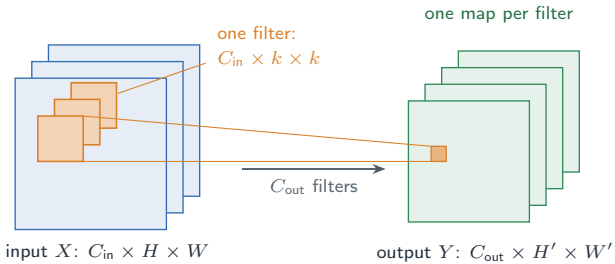
Our network (slide 11)

layer	k	s	r	j	r computed as
input			1	1	
conv 5×5	5	1	**5**	1	$1 + 4 \cdot 1$
max-pool 2×2	2	2	**6**	2	$5 + 1 \cdot 1$
conv 5×5	5	1	**14**	2	$6 + 4 \cdot 2$
max-pool 2×2	2	2	**16**	4	$14 + 1 \cdot 2$

A unit of the last feature map sees a 16×16 window of the 28×28 image: enough for a sleeve or a heel, not the whole garment. The linear layers after it combine all 25 positions.

- **Depth builds a hierarchy**: edges, then corners and textures, then parts [12].
- **Small kernels, stacked**: two 3×3 layers see 5×5 with $2 \cdot 9 = 18$ weights instead of 25, three see 7×7 with 27 instead of 49, and they add nonlinearities in between. This is the design rule of VGG [5].
- The *effective* receptive field is smaller than r: central pixels have many more paths to the unit than peripheral ones [16].

Many channels in, many filters out



The convolution layer in full

$$Y_{o,i,j} = b_o + \sum_{c=0}^{C_{\text{in}}-1} \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} W_{o,c,u,v}\, X_{c,\, si+u,\, sj+v}$$

Weight tensor $W \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k \times k}$, bias $b \in \mathbb{R}^{C_{\text{out}}}$; X is the padded input, s the stride. In PyTorch: `nn.Conv2d(C_in, C_out, k)`, whose `weight.shape` is $(C_{\text{out}}, C_{\text{in}}, k, k)$.

- A filter is a small **volume**: it spans *all* input channels but only $k \times k$ pixels. One output value is the dot product of that volume with the patch under it.
- Each filter yields one **feature map**. C_{out} filters give C_{out} maps, which are the input channels of the next layer.
- Typical pattern: spatial size goes down, channel count goes up.

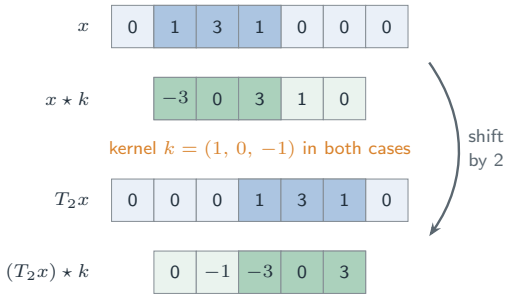
Counting parameters: $C_{\text{out}}\,(C_{\text{in}}\,k^2 + 1)$

our conv1	$(8, 1, 5, 5)$	$8\,(1 \cdot 25 + 1) = \mathbf{208}$
our conv2	$(16, 8, 5, 5)$	$16\,(8 \cdot 25 + 1) = \mathbf{3{,}216}$
RGB, 64 filters	$(64, 3, 3, 3)$	$64\,(3 \cdot 9 + 1) = \mathbf{1{,}792}$

The count does not depend on H or W. A dense layer from 784 pixels to the $8 \cdot 28 \cdot 28 = 6{,}272$ outputs of conv1 would need 4,923,520 parameters instead of 208.

1×1 **convolution**: $k = 1$ mixes channels at each pixel separately; it is a dense layer applied at every position, used to change the channel count cheaply [10].

Weight sharing and translation equivariance



Shifting the input by 2 shifts the output by 2 (dark cells). What is pushed past the border is lost (the 1 near the end of the first output); a new value enters on the other side.

Translation equivariance

$$(T_a x) \star k \;=\; T_a\,(x \star k) \qquad T_a\text{: shift by } a \text{ pixels}$$

Exact away from the border. It follows from the definition: the weights $K_{u,v}$ do not depend on the position (i, j).

- **Weight sharing**: every position uses the same kernel. A feature learned in one corner is detected everywhere, with no extra parameters or training examples.
- **Equivariant** is not **invariant**. Equivariant: the feature map moves with the object. Invariant: the answer does not change. Classification wants invariance; pooling and the head (slides 9, 10) must turn one into the other.

Measured on a real image (experiment C)

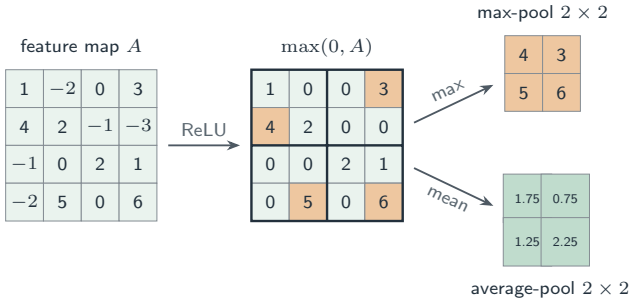
Test image 0 shifted 2 pixels down, 3 right; four random 5×5 kernels, padding 2.

$\max\lvert(Tx) \star K - T(x \star K)\rvert$, interior	**0**
same, including the border rows and columns	9.97
random dense layer, $\lVert W\,Tx - Wx\rVert / \lVert Wx\rVert$	83%

The interior agrees to the last bit. A dense layer has no output positions that could move: its response to the shifted image is just a different vector.

Caveat: with stride or pooling this holds only for shifts by multiples of the total stride, and a trained CNN is *not* shift-invariant [15]. Slide 14 measures it.

Nonlinearity and pooling



ReLU, $\max(0, a)$, applied to every value. Without a nonlinearity two convolutions in a row collapse into one larger convolution (linear after linear is linear). A zero means "this feature is absent here".

Pooling summarises each 2×2 block (kernel 2, stride 2) of each channel separately, by its maximum or its mean:

$$\text{pool}(A)_{i,j} = \max_{0 \le u,v < 2} A_{2i+u,\, 2j+v}\,.$$

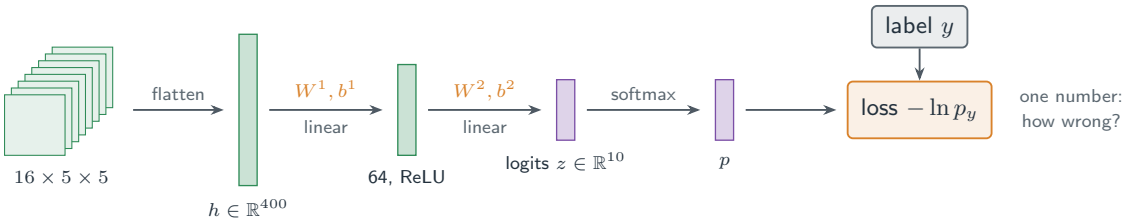
Output size (slide 5): $\lfloor (4-2)/2 \rfloor + 1 = 2$. No parameters.

- **Why pool?** It removes 75% of the values, so later layers are four times cheaper, and it doubles the jump j of slide 6: the receptive field grows faster.
- **A little invariance**: if the 4 in the top-left block moves to another cell of its block, the max-pooled output is unchanged. A first step from equivariance to invariance.
- **Max or average?** Max: "is the feature anywhere in the block?" Average: "how much of it?" Max is the usual choice inside a network.
- Many modern networks downsample with stride-2 convolutions instead [6].

Global average pooling (GAP) [10]

Average each channel over *all* positions: $C \times H \times W \to C$ numbers. For the map above: $24/16 = \mathbf{1.5}$. It replaces the large flatten-plus-dense head: with a GAP head our network has 3,594 parameters instead of 29,738 (`CNN(head="gap")`), and accepts any input size.

The classifier head: flatten, linear, softmax, cross-entropy

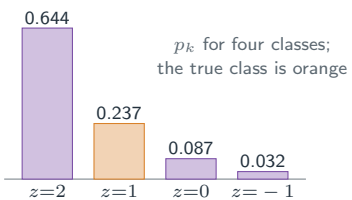


- **Flatten**: write the $16 \cdot 5 \cdot 5$ values as one vector of 400; an ordinary MLP follows.
- **Logits** $z = W^2\,\mathrm{ReLU}(W^1 h + b^1) + b^2$: one raw score per class.
- **Softmax** turns scores into probabilities:

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}, \qquad p_k > 0, \quad \sum_k p_k = 1.$$

- **Cross-entropy** loss for true class y:

$$\mathcal{L} = -\ln p_y, \qquad \frac{\partial \mathcal{L}}{\partial z} = p - \mathrm{onehot}(y).$$



p_k for four classes; the true class is orange

Untrained, all $p_k \approx \frac{1}{10}$: the loss starts near $\ln 10 = \mathbf{2.30}$.

By hand (4 classes)

$z = (2,\ 1,\ 0,\ -1)$

$e^z = (7.389,\ 2.718,\ 1,\ 0.368)$, sum 11.475

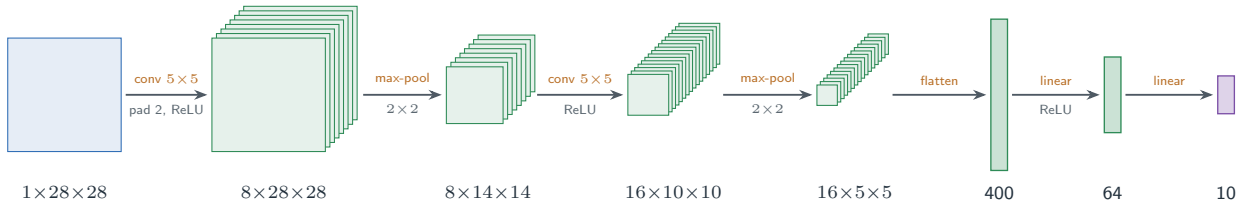
$p = (0.644,\ 0.237,\ 0.087,\ 0.032)$

True class is the 2nd:

$\mathcal{L} = -\ln 0.237 = \mathbf{1.440}$

$\partial \mathcal{L}/\partial z =$ $(0.644, -0.763, 0.087, 0.032)$: raise the true score, lower the rest.

The complete network: a LeNet-style CNN, shape by shape



layer	output (C, H, W)	values	parameters
input image	$1 \times 28 \times 28$	784	–
conv 5×5, pad 2, ReLU	$8 \times 28 \times 28$	6,272	208
max-pool 2×2	$8 \times 14 \times 14$	1,568	–
conv 5×5, pad 0, ReLU	$16 \times 10 \times 10$	1,600	3,216
max-pool 2×2	$16 \times 5 \times 5$	400	–
flatten	400	400	–
linear, ReLU	64	64	25,664
linear	10	10	650
total			**29,738**

Generated from `shape_table(CNN())` in `cnn.py`; the column sum is checked against the model.

- The pattern is LeNet's [2,3]: [convolution, nonlinearity, pooling] twice, then dense layers. LeNet-5 used tanh-like units and trainable subsampling instead of ReLU and max-pooling.
- **Parameters**: the two convolutions hold 3,424 (12%); the first dense layer alone holds 25,664.
- **Arithmetic**: conv2 does $1{,}600 \cdot 200 = 320{,}000$ multiplications per image, conv1 $6{,}272 \cdot 25 = 156{,}800$, both dense layers together $26{,}240$. Convolutions are cheap in parameters, not in computation.
- **Baselines** (slide 17): an MLP $784 \to 128 \to 64 \to 10$ with 109,386 parameters (3.7 times more) and softmax regression $784 \to 10$ with 7,850.

Backpropagation through a convolution

Forward: $Y = X \star K + b$. The layer above hands us $\delta = \partial\mathcal{L}/\partial Y$, one number per output position.

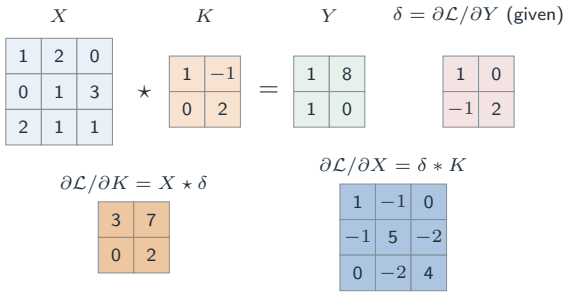
Backward pass of one convolution (stride 1)

$$\frac{\partial \mathcal{L}}{\partial K_{u,v}} = \sum_{i,j} \delta_{i,j}\, X_{i+u,\, j+v} \quad \Longleftrightarrow \quad \frac{\partial \mathcal{L}}{\partial K} = X \star \delta$$

$$\frac{\partial \mathcal{L}}{\partial X_{m,n}} = \sum_{u,v} \delta_{m-u,\, n-v}\, K_{u,v} \quad \Longleftrightarrow \quad \frac{\partial \mathcal{L}}{\partial X} = \delta * K$$

$$\frac{\partial \mathcal{L}}{\partial b} = \sum_{i,j} \delta_{i,j}$$

- **Kernel gradient**: the input correlated with the error map. The kernel is shared, so its gradient is a *sum over all positions*.
- **Input gradient**: a true (flipped, "full") convolution: pad δ by $k-1$ and correlate with K rotated by $180°$. This flows on to the layer below.
- **ReLU**: pass δ where the input was positive. **Max-pool**: send δ to the cell that held the maximum.



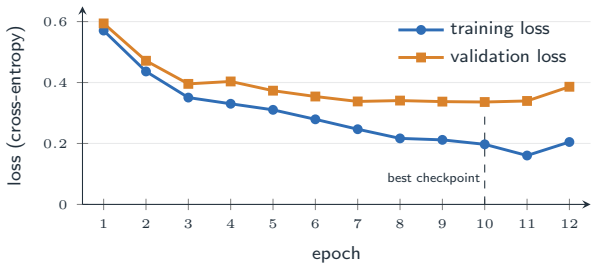
Check two entries. $\partial\mathcal{L}/\partial K_{0,0} = 1\cdot 1 + 0\cdot 2 + (-1)\cdot 0 + 2\cdot 1 = \mathbf{3}$ (δ times the top-left 2×2 patch of X). The centre pixel $X_{1,1}$ feeds all four outputs: $1\cdot 2 + 0\cdot 0 + (-1)(-1) + 2\cdot 1 = \mathbf{5}$.

Measured: `conv2d_backward` (experiment E)

vs. centred finite differences: worst relative error over x, w, b and three stride/padding settings	6×10^{-9}
vs. PyTorch autograd: worst absolute difference	5×10^{-15}

The code handles any stride and padding ($X_{si+u,\, sj+v}$ in the sums).

Training: mini-batches, optimizer, learning rate



Measured: CNN of slide 11, seed 0, Adam, $\eta = 0.002$, batch 64; training loss on 2,000 training images. One run.

- **Mini-batch SGD**: estimate the gradient on B images, step, repeat: $\theta \leftarrow \theta - \eta \nabla_\theta \frac{1}{B} \sum_{n=1}^{B} \mathcal{L}_n$. With $B = 64$, one **epoch** over 10,000 images is 157 steps.
- **Validation** after every epoch. Keep the weights of the best epoch (here 10, loss 0.34); stop after 3 epochs without improvement.
- The gap between the curves is **overfitting**: after epoch 12 the training loss is 0.20, the validation loss 0.39 (slide 14).



Measured: same CNN, 3,000 training images, 2 epochs, one run per point (experiment F). Values above 2.5 are drawn at 2.5.

- The **learning rate** η matters most. Too small: little happens in the time available. Too large: an early step throws the weights so far that every image gets the same answer; the loss stays at $\ln 10$.
- **Adam** [9] divides each weight's step by the root of a running average of its squared gradient. It reaches a loss below 1 here for η from 0.0003 to 0.01, plain SGD only from 0.1 to 0.3.
- Everywhere else: Adam, $\eta = 0.002$, no schedule.

Regularisation, normalisation, augmentation: what actually helped

Batch normalisation [7], after a convolution. Per channel c, with mean μ_c and std σ_c over the batch and all positions:

$$\hat{x} = \frac{x - \mu_c}{\sigma_c}, \qquad y = \gamma_c \hat{x} + \beta_c \qquad (\gamma_c, \beta_c \text{ learned})$$

Example: $(1, 3, 5, 7) \to \hat{x} = (-1.34, -0.45, 0.45, 1.34)$. Test time: running averages.

Dropout [8], before the last layer. In training, zero each hidden activation with probability $p = 0.3$, divide the rest by $1 - p$. Off at test time.

Data augmentation. Each time an image is used, shift it by a random (d_y, d_x), $|d_y|, |d_x| \le 2$ pixels: 25 variants per image, same label.

Honest reading of the measurements

- Batch norm and dropout did not improve test accuracy here (12 epochs, 10,000 images): rows 1–3 differ by less than the seed noise, ± 0.8.
- Augmentation *lowered* centred-test accuracy, shrank the train–validation gap and bought robustness to shifts.
- Without it, a 2-pixel shift takes the CNN from 88.2% to 78.0%, the MLP from 85.6% to 63.4%: neither is invariant [15].

CNN variant (seed 0, accuracy in %)	val. acc.	test acc.	test, 2-px shift	val − train loss
plain (slide 11)	88.2	88.0	78.1	0.14
+ batch norm	88.3	87.5	79.1	0.12
+ dropout 0.3	88.7	87.5	78.5	0.12
+ shifts ±2 px	87.3	86.7	**86.0**	0.06
all three	87.3	86.7	**86.3**	0.04



Measured (experiments G, H). First 2,500 test images, mean of the 4 shift directions. Dashed and dotted: mean of 3 seeds; solid: seed 0.

Code: convolution from scratch in NumPy (forward and backward)

conv_numpy.py: the forward pass of slide 7 and the backward pass of slide 12 (lines verbatim; three lines of shape bookkeeping omitted)

```python
def conv2d(x: np.ndarray, w: np.ndarray, b=None, stride: int = 1, padding: int = 0) -> np.ndarray:
    """y[n, o, i, j] = b[o] + sum_{c, u, v} w[o, c, u, v] * x_padded[n, c, i*s + u, j*s + v]"""
    s, p = stride, padding
    xp = np.pad(x, ((0, 0), (0, 0), (p, p), (p, p)))              # zeros around the border
    Ho, Wo = out_size(H, k, s, p), out_size(W, k, s, p)
    y = np.zeros((N, Co, Ho, Wo), dtype=x.dtype)
    for i in range(Ho):                                           # slide the window over the image
        for j in range(Wo):
            patch = xp[:, :, i * s:i * s + k, j * s:j * s + k]    # (N, C_in, k, k)
            y[:, :, i, j] = np.einsum("nckl,ockl->no", patch, w)  # one dot product per filter
    return y if b is None else y + b[None, :, None, None]
```

```python
def conv2d_backward(dy: np.ndarray, x: np.ndarray, w: np.ndarray, stride: int = 1, padding: int = 0):
    k, s, p = w.shape[2], stride, padding
    xp = np.pad(x, ((0, 0), (0, 0), (p, p), (p, p)))
    dxp, dw = np.zeros_like(xp), np.zeros_like(w)
    for i in range(dy.shape[2]):
        for j in range(dy.shape[3]):
            patch = xp[:, :, i * s:i * s + k, j * s:j * s + k]
            dw += np.einsum("no,nckl->ockl", dy[:, :, i, j], patch)   # shared kernel: every position adds its vote
            dxp[:, :, i * s:i * s + k, j * s:j * s + k] += np.einsum("no,ockl->nckl", dy[:, :, i, j], w)
    dx = dxp[:, :, p:p + x.shape[2], p:p + x.shape[3]]            # drop the gradient of the padding
    return dx, dw, dy.sum(axis=(0, 2, 3))
```

Checked (`python conv_numpy.py`, `test_cnn.py`): forward equals `torch.nn.functional.conv2d` to 4×10^{-15}; backward equals autograd to 5×10^{-15} and finite differences to 6×10^{-9}. The trained network run through these functions plus `pool2d` (`cnn_forward`) reproduces the PyTorch logits. The loops are for reading; `conv2d_windows` does the same without them.

Code: the model and the training loop in PyTorch

`cnn.py`: the network of slide 11 (`norm` is batch norm or nothing; `c1, c2 = 8, 16`)

```
        self.features = nn.Sequential(                    # input            (N,  1, 28, 28)
            nn.Conv2d(1, c1, kernel_size=5, padding=2),   # 28 + 4 - 5 + 1   (N,  8, 28, 28)
            norm(c1), nn.ReLU(), nn.MaxPool2d(2),         # 28 / 2           (N,  8, 14, 14)
            nn.Conv2d(c1, c2, kernel_size=5),             # 14 - 5 + 1       (N, 16, 10, 10)
            norm(c2), nn.ReLU(), nn.MaxPool2d(2),         # 10 / 2           (N, 16,  5,  5)
        )
            self.head = nn.Sequential(
                nn.Flatten(),                             # 16 * 5 * 5       (N, 400)
                nn.Linear(c2 * 5 * 5, hidden), nn.ReLU(), #                  (N, 64)
                nn.Dropout(dropout),                      # active in training only
                nn.Linear(hidden, n_out),                 # logits           (N, 10)
            )
```

`train.py`, inside `fit()` (augmentation and logging omitted). Afterwards the best weights are restored and `evaluate()` reports test accuracy, the confusion matrix and a one-class baseline.

```
    for epoch in range(1, epochs + 1):
        model.train()
        for xb, yb in batches(d["X_train"], d["y_train"], batch_size, rng):   # one epoch
            logits = model(torch.from_numpy(xb))                      # 1. forward pass   (B, 10)
            loss = F.cross_entropy(logits, torch.from_numpy(yb))      # 2. softmax + negative log-likelihood
            opt.zero_grad()
            loss.backward()                                           # 3. backpropagation
            opt.step()                                                # 4. update every weight
        train_loss, train_acc = loss_and_accuracy(model, d["X_train"][:2000], d["y_train"][:2000])
        val_loss, val_acc = loss_and_accuracy(model, d["X_val"], d["y_val"])
        if val_loss < best_val - 1e-5:
            best_val, bad_epochs = val_loss, 0
            best_state = copy.deepcopy(model.state_dict())            # best checkpoint so far
```

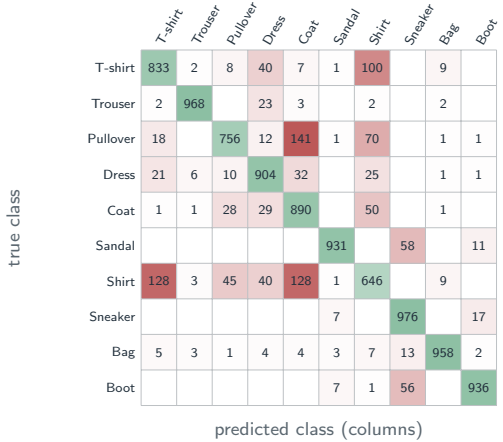
Results: CNN vs. MLP vs. softmax regression

`python train.py --model {cnn,mlp,linear} --seed {0,1,2}`

model	parameters	test accuracy mean ± std	per seed
CNN (slide 11)	29,738	**87.4** ± 0.8	88.0, 86.5, 87.6
MLP 128, 64	109,386	84.7 ± 0.1	84.8, 84.7, 84.7
softmax regression	7,850	82.8 ± 0.4	83.2, 82.4, 82.8
always one class	0	10	

Fashion-MNIST, 10,000 of the 60,000 training images, all 10,000 test images; accuracy in %. 3 seeds (a seed sets the train/validation split and the initial weights). Same optimizer settings for all. Seed-0 training time, one CPU core: 29 s, 3 s, 1 s.

- The CNN beats the MLP by 2.6 points with 3.7 times fewer parameters (17% of the MLP's errors removed). The MLP beats the linear model by 1.9.
- The gain is clear but modest, and the CNN has the largest seed-to-seed spread (three seeds give only a rough estimate of it).
- The price is computation (slide 11). This is a small-data, 12-epoch, CPU-sized setting; we did not train on all 60,000 images.

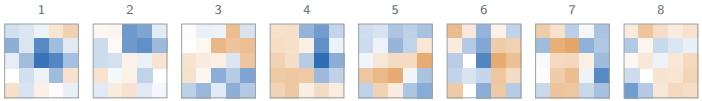


Confusion matrix, seed-0 CNN, 10,000 test images (1,000 per class).

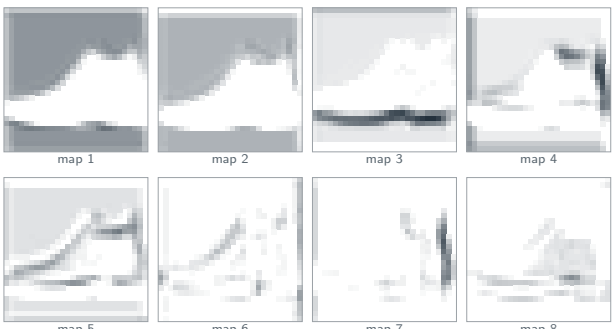
Where it fails. Most confused pairs: T-shirt / Shirt (228 errors), Coat / Shirt (178), Pullover / Coat (169). 78% of all errors are among the five upper-body garments. Recall ranges from 65% (Shirt) to 98% (Sneaker).

What the network learned: first-layer filters and feature maps

The 8 learned 5×5 kernels of conv1 (seed-0 CNN; orange > 0, blue < 0)



Their feature maps for test image 0, after ReLU (darker = larger)





the input: test image 0, true class Boot, predicted Boot

- Nobody designed these kernels: they began as random numbers and were shaped by gradient descent on 10,000 labelled images (experiment J; ordered by mean activation on this image).
- 44% of the values in the eight maps are exactly zero after ReLU: a map marks *where* its pattern occurs and is silent elsewhere.
- In this run, maps 1 and 2 fire on the empty background, map 3 on the sole (a horizontal edge), maps 4 and 7 on the vertical edge at the back of the shaft.
- The kernels are noisier than textbook pictures; the familiar clean oriented-edge filters come from much larger networks and datasets [4,12]. Deeper layers combine these maps into detectors for larger parts.

Four networks that shaped the field

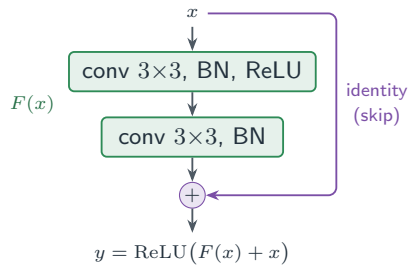
Neocognitron 1980 [1] → LeNet 1989–1998 [2,3] → AlexNet 2012 [4] → VGG 2014 [5] → ResNet 2015 [6] → Vision Transformer 2020 [18]

no convolution; needs very large datasets

network	year	weight layers	parameters	ImageNet top-5 error
LeNet-5 [3]	1998	7	60 thousand	n/a (MNIST digits)
AlexNet [4]	2012	8	60 million	15.3%
VGG-16 [5]	2014	16	138 million	7.3%
ResNet-152 [6]	2015	152	60 million	3.57%

Layers counted as in each paper. Top-5 test error of the ImageNet competition entries (ensembles). Quoted, not measured here.

- **LeNet**: convolution, subsampling and dense layers trained end to end by backpropagation, to read handwritten digits. The template for slide 11.
- **AlexNet**: the same idea scaled up, with ReLU, dropout, augmentation and GPUs. Its 2012 ImageNet win started the deep-learning era in vision.
- **VGG**: depth matters. Only 3×3 convolutions, stacked (slide 6), and 2×2 max-pooling.
- **ResNet**: residual connections make very deep networks trainable; batch norm after every convolution; global average pooling as the head.



Residual connection

Very deep plain networks had *higher training error* than shallower ones [6]: an optimisation failure, not overfitting. A residual block learns only the change $F(x)$ to its input. Doing nothing ($F = 0$) is easy, and gradients reach early layers directly through the skip path.

References and how to run the code

[1] Fukushima. *Neocognitron: A Self-organizing Neural Network Model for a Mechanism of Pattern Recognition Unaffected by Shift in Position.* Biological Cybernetics 36, 1980.

[2] LeCun, Boser, Denker, Henderson, Howard, Hubbard, Jackel. *Backpropagation Applied to Handwritten Zip Code Recognition.* Neural Computation 1(4), 1989.

[3] LeCun, Bottou, Bengio, Haffner. *Gradient-Based Learning Applied to Document Recognition.* Proceedings of the IEEE 86(11), 1998.

[4] Krizhevsky, Sutskever, Hinton. *ImageNet Classification with Deep Convolutional Neural Networks.* NeurIPS 2012.

[5] Simonyan, Zisserman. *Very Deep Convolutional Networks for Large-Scale Image Recognition.* ICLR 2015. arXiv:1409.1556

[6] He, Zhang, Ren, Sun. *Deep Residual Learning for Image Recognition.* CVPR 2016. arXiv:1512.03385

[7] Ioffe, Szegedy. *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift.* ICML 2015. arXiv:1502.03167

[8] Srivastava, Hinton, Krizhevsky, Sutskever, Salakhutdinov. *Dropout: A Simple Way to Prevent Neural Networks from Overfitting.* JMLR 15, 2014.

[9] Kingma, Ba. *Adam: A Method for Stochastic Optimization.* ICLR 2015. arXiv:1412.6980

[10] Lin, Chen, Yan. *Network In Network.* ICLR 2014. arXiv:1312.4400

[11] Dumoulin, Visin. *A Guide to Convolution Arithmetic for Deep Learning.* 2016. arXiv:1603.07285

[12] Zeiler, Fergus. *Visualizing and Understanding Convolutional Networks.* ECCV 2014. arXiv:1311.2901

[13] Xiao, Rasul, Vollgraf. *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms.* 2017. arXiv:1708.07747. Data: github.com/zalandoresearch/fashion-mnist

[14] Goodfellow, Bengio, Courville. *Deep Learning*, chapter 9. MIT Press, 2016. deeplearningbook.org

[15] Azulay, Weiss. *Why Do Deep Convolutional Networks Generalize so Poorly to Small Image Transformations?* JMLR 20, 2019. arXiv:1805.12177

[16] Luo, Li, Urtasun, Zemel. *Understanding the Effective Receptive Field in Deep Convolutional Neural Networks.* NeurIPS 2016. arXiv:1701.04128

[17] Sobel, Feldman. *A 3x3 Isotropic Gradient Operator for Image Processing.* Talk, Stanford Artificial Intelligence Project, 1968.

[18] Dosovitskiy et al. *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale.* ICLR 2021. arXiv:2010.11929

[19] Paszke et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library.* NeurIPS 2019. arXiv:1912.01703

Run it (Python, NumPy, PyTorch; CPU only)

`python test_cnn.py`	seven checks, incl. backward pass vs. finite differences
`python walkthrough.py`	the hand calculations (slides 2–10, 12, 14)
`python conv_numpy.py`	from-scratch convolution vs. PyTorch
`python train.py`	the harness, about a minute; `--help` lists every knob
`python experiments.py`	all figures and numbers, about 6 minutes

Exercises. Train `--model cnn_gap` and compare parameters and accuracy. Replace max- by average-pooling. Test the augmented CNN at a 6-pixel shift.